A

Dissertation Report

ON

"UK Housing Prices Data Analysis, Visualization & Prediction "

Submitted By:

[220340325038] Ranjit Kailasrao More (PL)

[220340325033] Prathmesh Shivaji Dalve

[220340325018] Jayraj Shrikant Maghade

[220340325029] Nayan Surendra Dhoble

[220340325050] Sonali Ashok Waman

[220340325046] Shivaraj Hiraman Shelar

[PG-DBDA]

C-DAC Kharghar

Parag Thakur
Project Guide

Vineeta Singh
Course Co-ordinator

Abstract

We have done data analysis, data visualisation and prediction of data in this project. The dataset we used is of housing prices paid of UK which we gathered from the HM Land Registry.

The whole project is based on cloud platform and Power BI. For performing big data analytics we used AWS's EMR service. We used Hive and PySpark by creating an EMR cluster. For data visualisation we used Power BI desktop. We used various visuals like bar charts, donut charts, cards, map, slicers, etc. and gathered some insights about the data.

For machine learning we used AWS's SageMaker. In which we created a cluster and created a python notebook in Jupyter Lab. We did EDA on the data and after that we did prediction using some machine learning algorithms available in python libraries.

This whole process and the insights about the data are explained in detail in this report.

Contents

Chapter I : INTRODUCTION

1.1 Introduction to the project work

In this project we downloaded the 'Price Paid Data' of all registered property sales in England and Wales that are sold for full market value. We used EMR(Elastic Map-Reduce) service from AWS(Amazon Web Services). In EMR we created a Hadoop cluster and first used Hive for data analysis and then PySpark for the same purpose. By analysing the data we gathered some insights about it and we have described the same in this project report. For visualising the same analysed data we used Microsoft Power BI Desktop and created interactive dashboards. We also did prediction using the same dataset. For that we used various machine learning algorithms available in python libraries. The predicted data and information are described in the report.

1.2 Introduction to dataset

The Price Paid Data includes information on all registered property sales in England and Wales that are sold for full market value. Address details have been truncated to the town/city level.

The available fields are as follows:

Transaction unique identifier A reference number which is generated automatically recording each published sale. The number is unique and will change each time a sale is recorded.

Price Sale price stated on the transfer deed.

Date of Transfer Date when the sale was completed, as stated on the transfer deed.

Property Type :

D = Detached, S = Semi-Detached, T = Terraced, F = Flats/Maisonettes, O = Other

Old/New Indicates the age of the property and applies to all price paid transactions, residential and non-residential.
Y = a newly built property, N = an established residential building

Duration Relates to the tenure: F = Freehold, L= Leasehold etc.

PPD Category Type Indicates the type of Price Paid transaction :
A = Standard Price Paid entry, includes single residential property sold for full market value.
B = Additional Price Paid entry including transfers under a power of sale/repossessions, buy-to-lets (where they can be identified by a Mortgage) and transfers to non-private individuals. Note that category B does not separately identify the transaction types stated. HM Land Registry has been collecting information on Category A transactions from January 1995. Category B transactions were identified from October 2013.

Record Status - monthly file only Indicates additions, changes and deletions to the records.(see guide below).
A = Addition
C = Change
D = Delete.

1.3 Problem Statement

Our goal is to analyse the prices paid using the given dataset. We have to categorise the price paid based on year, country, district and town or city. We also have to visualise the data based on the same constraints.

The future data is also to be predicted and the error is to be checked. We must apply various machine learning algorithms for the same.

1.4 Objectives

- Analysing the data according to price paid yearly
- Analysing the prices paid based on country, district and town
- Visualizing the data for better understanding
- Creating interactive dashboards
- Predicting the future data
- Calculating the error in predicted data

Chapter II : BIG DATA ANALYTICS

2.1 Data Collection

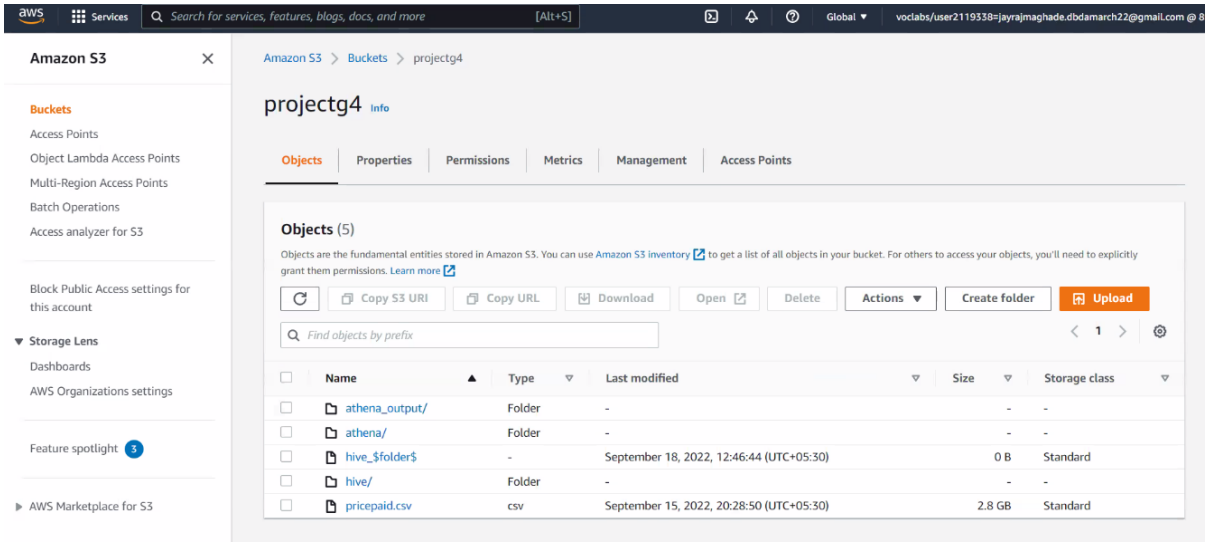
2.1.1 Dataset Source:

His Majesty's (HM) Land Registry's website

2.1.2 S3 Bucket Creation:

To store the data, we need a storage unit, for that we are using S3 bucket. S3 stands for Simple Storage Service, S3 stores data in object form.

We created an S3 bucket named "**projectg4**".



Users can upload data from local machines or EC2 instances. In our case we are uploading data from an EC2 instance.

2.1.3 EC2 Instance Creation :

Amazon EC2 stands for Elastic Compute Cloud. It is used for creating virtual machines for various projects.

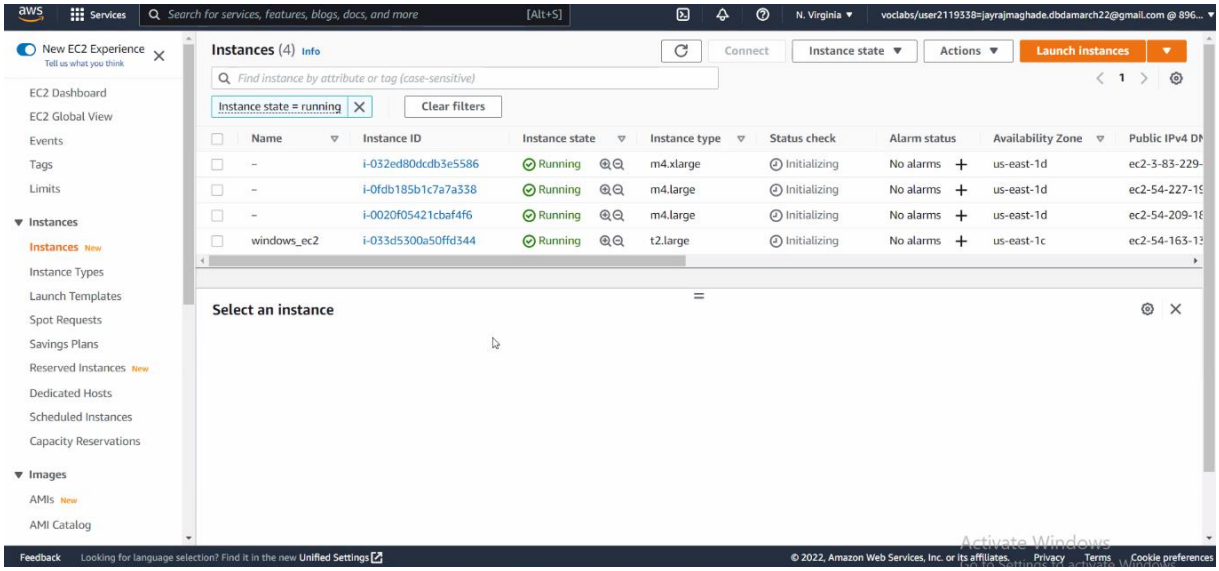
Using AWS's EC2 service, we created t2.large Windows instance with the specification 16 GB RAM, 40 GB storage.

Using the Remote Desktop Protocol Client (RDP Client) we launched an instance.

Using internet service, we downloaded data from UK Land Registry Transactions.

After that, on AWS CLI we uploaded data into S3 bucket, command used for that is :

- aws s3 cp C:\Project s3://projectg4



2.2 Creating EMR cluster

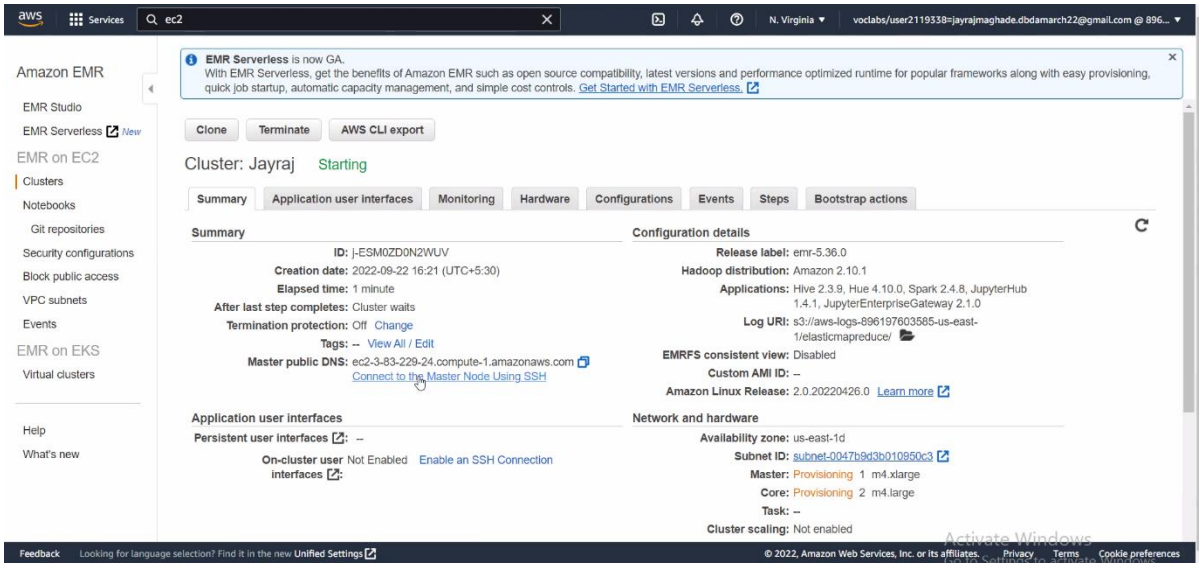
EMR stands for Elastic MapReduce, which is a cluster and provides Big Data frameworks, such as hadoop, spark, PySpark etc. These Frameworks are used for data processing.

EMR is an alternative to on premise cluster computing. Users can scale up or scale down as per requirements.

For project purpose we created an EMR with specifications:

'm4.xlarge' for master node, which has 4 cores, 16 GB RAM and 64 GB storage.

And 2 slave nodes, each node had 'm4.large', with 2 cores, 8 GB RAM and 32 GB storage.



In EMR, we used frameworks such as Hive, Spark and PySpark.

2.3 Analysis using Spark SQL

Spark SQL is a Spark module for structured data processing. It provides a programming abstraction called DataFrames and can also act as a distributed SQL query engine. It enables unmodified Hadoop Hive queries to run up to 100x faster on existing deployments and data.

Spark SQL brings native support for SQL to Spark and streamlines the process of querying data stored both in RDDs (Spark's distributed datasets) and in external sources. Spark SQL conveniently blurs the lines between RDDs and relational tables.

Steps to create a table in PySpark –

1. Launch PySpark
2. Import required datatypes **e.g.** `from pyspark.sql.types import IntegerType`
3. Creating a schema with required column names and their datatypes.

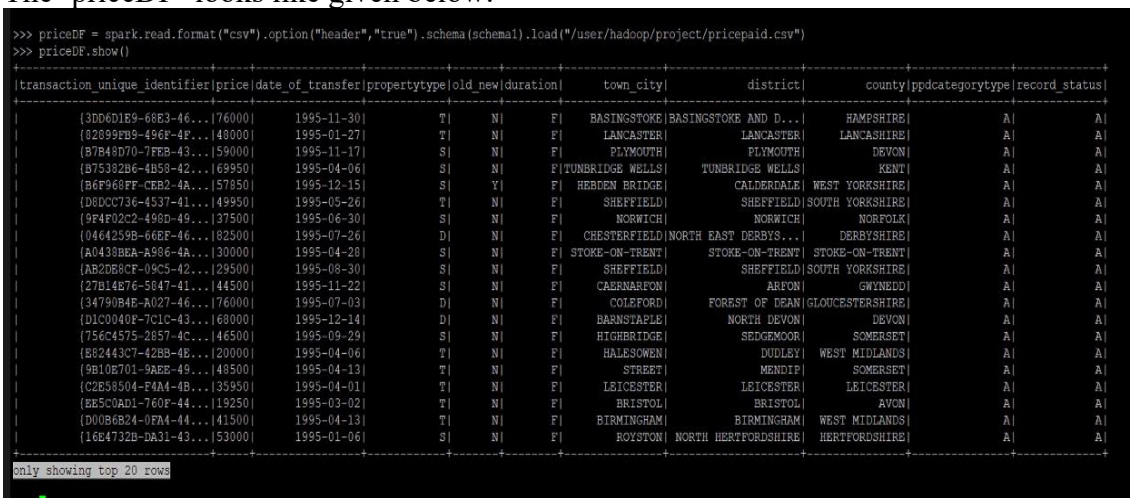
```
Schema1   =   StructType().add("transaction_unique_identifier",
StringType(),True)
.add("price",       IntegerType(),True).add("date_of_transfer",
DateType(), True)
.add("propertytype",     StringType(),     True).add("old_new",
StringType() , True)
.add("duration",      StringType(),      True).add("town_city",
StringType(),True)
.add("district",               StringType(),True).add("county",
StringType(),True)
.add("ppdcategorytype" ,StringType(),True).add("record_status",
StringType(),True)
```

[Here True means if the column allows null values, true for nullable, and false for not nullable.]

After creating schema we can perform some actions such as –

- pricedf.printSchema() – for describing the schema.
- pricedf.show() – for displaying first 20 rows.
- pricedf.count() – for counting number of rows.

4. Reading the data from .csv file –
```
pricedf =
spark.read.format("csv").option("header","true").schema(schema
1).load("/user/hadoop/project/pricepaid.csv")
```
The 'priceDF' looks like given below:

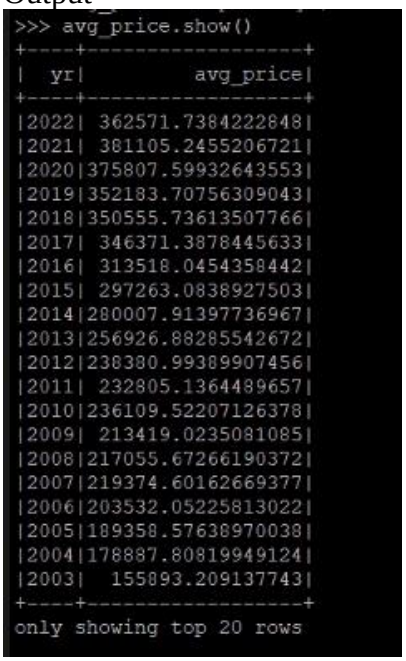


```
>>> priceDF = spark.read.format("csv").option("header","true").schema(schema1).load("/user/hadoop/project/pricepaid.csv")
>>> priceDF.show()
+------------------------------+-----+----------------+------------+-------+--------+--------------+--------------------+--------------+---------------+-------------+
|transaction_unique_identifier|price|date_of_transfer|propertytype|old_new|duration|     town_city|            district|        county|ppdcategorytype|record_status|
+------------------------------+-----+----------------+------------+-------+--------+--------------+--------------------+--------------+---------------+-------------+
|        {3DD6D1E9-68B3-46...|76000|      1995-11-30|           T|      N|       F|   BASINGSTOKE|BASINGSTOKE AND D...|     HAMPSHIRE|              A|            A|
|        {82899FB9-496F-4F...|48000|      1995-01-27|           T|      N|       F|     LANCASTER|            LANCASTER|    LANCASHIRE|              A|            A|
|        {B7B48D70-7FEB-43...|59000|      1995-11-17|           S|      N|       F|      PLYMOUTH|            PLYMOUTH|         DEVON|              A|            A|
|        {B75382B6-4B58-42...|69950|      1995-04-06|           S|      N|       F|TUNBRIDGE WELLS|     TUNBRIDGE WELLS|          KENT|              A|            A|
|        {B6F968FF-CEB2-4A...|57850|      1995-12-15|           S|      Y|       F| HEBDEN BRIDGE|          CALDERDALE| WEST YORKSHIRE|              A|            A|
|        {D8DCC736-4537-41...|49950|      1995-05-26|           T|      N|       F|     SHEFFIELD|           SHEFFIELD|SOUTH YORKSHIRE|              A|            A|
|        {9F4F02C2-498D-49...|37500|      1995-06-30|           S|      N|       F|       NORWICH|              NORWICH|       NORFOLK|              A|            A|
|        {0464259B-66EF-46...|82500|      1995-07-26|           D|      N|       F|CHESTERFIELD|NORTH EAST DERBYS...|    DERBYSHIRE|              A|            A|
|        {A0438BEA-A986-4A...|30000|      1995-04-28|           S|      N|       F|STOKE-ON-TRENT|      STOKE-ON-TRENT| STOKE-ON-TRENT|              A|            A|
|        {AB2DB8CF-09C5-42...|29500|      1995-08-30|           S|      N|       F|     SHEFFIELD|           SHEFFIELD|SOUTH YORKSHIRE|              A|            A|
|        {27B14B76-5847-41...|44500|      1995-11-22|           S|      N|       F|    CAERNARFON|              ARFON|       GWYNEDD|              A|            A|
|        {34790B4E-A027-46...|76000|      1995-07-03|           D|      N|       F|      COLEFORD|      FOREST OF DEAN|GLOUCESTERSHIRE|              A|            A|
|        {D1C0040F-7C1C-43...|68000|      1995-12-14|           D|      N|       F|    BARNSTAPLE|         NORTH DEVON|         DEVON|              A|            A|
|        {756C4575-2857-4C...|46500|      1995-09-29|           S|      N|       F|    HIGHBRIDGE|           SEDGEMOOR|       SOMERSET|              A|            A|
|        {E82443C7-42BB-4E...|20000|      1995-04-06|           T|      N|       F|     HALESOWEN|              DUDLEY|  WEST MIDLANDS|              A|            A|
|        {9B10E701-9AEE-49...|48500|      1995-04-13|           T|      N|       F|        STREET|             MENDIP|       SOMERSET|              A|            A|
|        {C2E58504-F4A4-4B...|35950|      1995-04-01|           T|      N|       F|     LEICESTER|           LEICESTER|      LEICESTER|              A|            A|
|        {EE5C0AD1-760F-44...|19250|      1995-03-02|           T|      N|       F|       BRISTOL|             BRISTOL|         AVON|              A|            A|
|        {D00B6B24-0FA4-44...|41500|      1995-04-13|           T|      N|       F|    BIRMINGHAM|          BIRMINGHAM|  WEST MIDLANDS|              A|            A|
|        {16E4732B-DA31-43...|53000|      1995-01-06|           S|      N|       F|       ROYSTON| NORTH HERTFORDSHIRE|  HERTFORDSHIRE|              A|            A|
+------------------------------+-----+----------------+------------+-------+--------+--------------+--------------------+--------------+---------------+-------------+
only showing top 20 rows
```

5. For performing SQL queries on a schema we have to create a temporary table. This temporary table is automatically dropped after the session ends. To register a 'temptable' - `priceDF.registerTempTable("pricepaid")`

After registering temp table we can perform the SQL queries on the table. We had to answer some business questions listed below. We have answered first 10 of them using Spark SQL.

1. Yearly average price paid per year
```
avg_price = spark.sql("select year(date of transfer) as yr
,avg(price) as price from pricepaid group by yr order by yr
desc")
avg_price.show()
```
Output –



```
>>> avg_price.show()
+----+------------------+
|  yr|         avg_price|
+----+------------------+
|2022| 362571.7384222848|
|2021| 381105.2455206721|
|2020|375807.59932643553|
|2019|352183.70756309043|
|2018|350555.73613507766|
|2017| 346371.3878445633|
|2016| 313518.0454358442|
|2015| 297263.0838927503|
|2014|280007.91397736967|
|2013|256926.88285542672|
|2012|238380.99389907456|
|2011| 232805.1364489657|
|2010|236109.52207126378|
|2009| 213419.0235081085|
|2008|217055.67266190372|
|2007|219374.60162669377|
|2006|203532.05225813022|
|2005|189358.57638970038|
|2004|178887.80819949124|
|2003|  155893.209137743|
+----+------------------+
only showing top 20 rows
```

2. In which year the average price was highest?

```
max_avg = spark.sql("select year(date_of_transfer) as yr, avg(price) as avg_price from pricepaid group by yr order by avg_price desc limit 1")
```

```
>>> max_avg.show()
+----+-----------------+
|  yr|        avg_price|
+----+-----------------+
|2021|381105.2455206721|
+----+-----------------+
```

Here we can see that the year 2021 had the highest average price paid of all years which is £381.105K.

3. What was the best month for sales in all years? What was the max price in that month?

```
month = spark.sql("select month(date_of_transfer) as month_no, sum(price) as total_price from pricepaid group by month_no order by total_price desc")
```

```
>>> month.show()
+--------+------------+
|month_no| total_price|
+--------+------------+
|       6|553772871262|
|       8|532242800350|
|      12|525987823681|
|       7|525054783892|
|       9|522685297859|
|       3|508409831809|
|      10|506016730965|
|      11|505439067397|
|       5|449154554686|
|       4|424242073664|
|       2|392800969266|
|       1|384974953661|
+--------+------------+
```

The month of June was discovered to be the best of all with highest price paid that sums up to be around £553.7 Billion, followed by August with £532.24 billion.

4. Highest price paid in each year

```
sum_year = spark.sql("select year(date of transfer) as yr, sum(price) as total_price from pricepaid group by yr order by total_price desc")
sum_year.show()
```

```
+----+------------+
|  yr| total_price|
+----+------------+
|2021|423809231597|
|2017|369001216098|
|2018|362624318463|
|2019|353939695529|
|2016|327671504079|
|2020|324497084560|
|2015|300318353869|
|2007|279050635758|
|2014|275673391469|
|2006|269835671262|
|2004|220320550120|
|2013|208314518131|
|2005|200940504356|
```

The year 2021 had the highest total price paid and that is £423.80 billion and the year 2017 was found to be second highest with £369 billion.

5. City wise sales.

```
town_total = spark.sql("select town_city, sum(price) as total_price from pricepaid group by town_city order by total_price desc")
town.show()
```

It was found out that the London City had the most price paid in every year with highest price paid in year 2021 which was £70.7 billion.

6. District wise sales -

```
district_total = spark.sql("select district, sum(price) as total_price from pricepaid group by district order by total_price desc")
district_total.show()
```

Westminster district had the highest total price paid about £132 billion followed by Kensington and Chelsea with £93.8 billion

7. County wise sales –

```
county_total = spark.sql("select county, sum(price) as total_price from pricepaid group by county order by total_price desc")
county_total.show()
```

```
>>> county_total.show()
+-----------------+-------------+
|           county|  total_price|
+-----------------+-------------+
|    GREATER LONDON|1390447199741|
|           SURREY| 221418939624|
|        HAMPSHIRE| 177225805776|
|             KENT| 176305817771|
|            ESSEX| 175777307125|
|GREATER MANCHESTER| 175197525731|
|     HERTFORDSHIRE| 169409650929|
|     WEST MIDLANDS| 158290376286|
|    WEST YORKSHIRE| 144515303854|
|       WEST SUSSEX| 117991273185|
|       OXFORDSHIRE|  94353769697|
|            DEVON|  93109750157|
|   BUCKINGHAMSHIRE|  86749576679|
|          NORFOLK|  81502233745|
|       LANCASHIRE|  80565059424|
|       MERSEYSIDE|  77606285478|
|          SUFFOLK|  73523631775|
|    CAMBRIDGESHIRE|  73254709390|
|   SOUTH YORKSHIRE|  72235508856|
|   GLOUCESTERSHIRE|  71916674617|
+-----------------+-------------+
```

We found out that the Greater London has the highest total price paid of all the counties which is about £1.39 Trillion. This price paid is about 21% of total price paid in UK. We can say that there is a major part of Greater London in the UK Housing Market.

8. Which type of houses are preferred most?

```
house_type = spark.sql("select propertytype, count(*) as total from pricepaid group by propertytype order by total desc")
house_type.show()
```

```
>>> house_type.show()
+------------+-------+
|propertytype|  total|
+------------+-------+
|           T|8260439|
|           S|7522784|
|           D|6333026|
|           F|4929023|
|           O| 405227|
+------------+-------+
```

There are four types of known property types – Terraced, Semi-Detached, Detached, Flats and the fifth is other. Out of all these the terraced houses are most preferred in UK which have count of 8.2 million and the second preferred are semi-detached houses with count 7.52 million.

9. Which type of property is sold most between freehold and leasehold.

```
hold = spark.sql("select duration, count(*) as total from pricepaid group by duration order by total desc")
```

```
>>> hold.show()
+--------+--------+
|duration|   total|
+--------+--------+
|       F|20996240|
|       L| 6453726|
|       U|     533|
+--------+--------+
```

We can observe that the people prefer to buy properties on freehold more than leasehold. Out of total 27 million properties, the total count of freehold properties is 20.99 million.

10. How many new properties are sold till now?

```
new = spark.sql("select old_new as new, count(*) as total from pricepaid where old_new = 'Y' group by old_new")
new.show()
```

We could see the count of new properties sold till the date is more than 2.81 million.

2.4 Analysis using PySpark

PySpark is a Python API for Apache Spark to process datasets in a distributed cluster. It is written in Python to run a Python application using Apache Spark Capabilities.

PySpark is very well used in Data Science and Machine Learning community as there are many widely used data science libraries written in Python.

PySpark has modules such as , Resilient Distributed Dataset (RDD), Dataframe, Pyspark streaming, MLib etc.

Operations :

For performing PySpark operations first we created RDD. Syntax for RDD is :

rdd1 = sc.textFile("/user/hadoop/project/pricepaid.csv")

Then we removed header, for that we performed following operations :-

header = rdd1.first()
rdd2 = rdd1.filter(lambda a : a != header)

```
SparkSession available as 'spark'.
>>> rdd1 = sc.textFile("/user/hadoop/project/pricepaid.csv")
>>> for i in rdd1.take(10):
...     print(i)
...
Transaction unique identifier,Price,Date of Transfer,Property Type,Old/New,Duration,Town/City,District,County,PPDCategory Type,Record Status
{3DD6D1E9-68E3-4646-8682-FD7DB9884D97},76000,1995-11-30 00:00,T,N,F,BASINGSTOKE,BASINGSTOKE AND DEANE,HAMPSHIRE,A,A
{82899FB9-496F-4FA3-8249-F6811A87F376},48000,1995-01-27 00:00,T,N,F,LANCASTER,LANCASTER,LANCASHIRE,A,A
{B7B48D70-7FEB-437B-9EAC-F681547AB62F},59000,1995-11-17 00:00,S,N,F,PLYMOUTH,PLYMOUTH,DEVON,A,A
{B75382B6-4B58-4284-A890-FD7E1C297E5D},69950,1995-04-06 00:00,S,N,F,TUNBRIDGE WELLS,TUNBRIDGE WELLS,KENT,A,A
{B6F968FF-CEB2-4A18-B212-EF6CFE08958D},57850,1995-12-15 00:00,S,Y,F,HEBDEN BRIDGE,CALDERDALE,WEST YORKSHIRE,A,A
{D8DCC736-4537-41E2-A821-EF6CFF2551A9},49950,1995-05-26 00:00,T,N,F,SHEFFIELD,SHEFFIELD,SOUTH YORKSHIRE,A,A
{9F4F02C2-498D-492E-918E-EF6D00C8B9BB},37500,1995-06-30 00:00,S,N,F,NORWICH,NORWICH,NORFOLK,A,A
{0464259B-66EF-4681-89DD-F2F06FD19F4F},82500,1995-07-26 00:00,D,N,F,CHESTERFIELD,NORTH EAST DERBYSHIRE,DERBYSHIRE,A,A
{A0438BEA-A986-4A49-B03B-F2F0827E52B7},30000,1995-04-28 00:00,S,N,F,STOKE-ON-TRENT,STOKE-ON-TRENT,STOKE-ON-TRENT,A,A
>>> hdr = rdd1.first()
>>> rdd2 = rdd1.filter(lambda a : a!= hdr)
>>> for i in rdd2.take(10):
...     print(i)
...
{3DD6D1E9-68E3-4646-8682-FD7DB9884D97},76000,1995-11-30 00:00,T,N,F,BASINGSTOKE,BASINGSTOKE AND DEANE,HAMPSHIRE,A,A
{82899FB9-496F-4FA3-8249-F6811A87F376},48000,1995-01-27 00:00,T,N,F,LANCASTER,LANCASTER,LANCASHIRE,A,A
{B7B48D70-7FEB-437B-9EAC-F681547AB62F},59000,1995-11-17 00:00,S,N,F,PLYMOUTH,PLYMOUTH,DEVON,A,A
{B75382B6-4B58-4284-A890-FD7E1C297E5D},69950,1995-04-06 00:00,S,N,F,TUNBRIDGE WELLS,TUNBRIDGE WELLS,KENT,A,A
{B6F968FF-CEB2-4A18-B212-EF6CFE08958D},57850,1995-12-15 00:00,S,Y,F,HEBDEN BRIDGE,CALDERDALE,WEST YORKSHIRE,A,A
{D8DCC736-4537-41E2-A821-EF6CFF2551A9},49950,1995-05-26 00:00,T,N,F,SHEFFIELD,SHEFFIELD,SOUTH YORKSHIRE,A,A
{9F4F02C2-498D-492E-918E-EF6D00C8B9BB},37500,1995-06-30 00:00,S,N,F,NORWICH,NORWICH,NORFOLK,A,A
{0464259B-66EF-4681-89DD-F2F06FD19F4F},82500,1995-07-26 00:00,D,N,F,CHESTERFIELD,NORTH EAST DERBYSHIRE,DERBYSHIRE,A,A
{A0438BEA-A986-4A49-B03B-F2F0827E52B7},30000,1995-04-28 00:00,S,N,F,STOKE-ON-TRENT,STOKE-ON-TRENT,STOKE-ON-TRENT,A,A
{AB2DE8CF-09C5-422A-9A30-F2F09709F4B7},29500,1995-08-30 00:00,S,N,F,SHEFFIELD,SHEFFIELD,SOUTH YORKSHIRE,A,A
```

As our dataset file is in CSV format, we need to split file on the basis of comma.

rdd3 = rdd2.map(lambda a : a.split(“,”))

After splitting the data looks like this:

```
>>> rdd3 = rdd1.map(lambda a : a.split(","))
>>> for i in rdd3.take(10):
...     print(i)
...
['Transaction unique identifier', 'Price', 'Date of Transfer', 'Property Type', 'Old/New', 'Duration', 'Town/City', 'District', 'County', 'PPDCategory Type',
'Record Status']
['{3DD6D1E9-68E3-4646-8682-FD7DB9884D97}', '76000', '1995-11-30 00:00', 'T', 'N', 'F', 'BASINGSTOKE', 'BASINGSTOKE AND DEANE', 'HAMPSHIRE', 'A', 'A']
['{82899FB9-496F-4FA3-8249-F6811A87F376}', '48000', '1995-01-27 00:00', 'T', 'N', 'F', 'LANCASTER', 'LANCASTER', 'LANCASHIRE', 'A', 'A']
['{B7B48D70-7FEB-437B-9EAC-F681547AB62F}', '59000', '1995-11-17 00:00', 'S', 'N', 'F', 'PLYMOUTH', 'PLYMOUTH', 'DEVON', 'A', 'A']
['{B75382B6-4B58-4284-A890-FD7E1C297E5D}', '69950', '1995-04-06 00:00', 'S', 'N', 'F', 'TUNBRIDGE WELLS', 'TUNBRIDGE WELLS', 'KENT', 'A', 'A']
['{B6F968FF-CEB2-4A18-B212-EF6CFE08958D}', '57850', '1995-12-15 00:00', 'S', 'Y', 'F', 'HEBDEN BRIDGE', 'CALDERDALE', 'WEST YORKSHIRE', 'A', 'A']
['{D8DCC736-4537-41E2-A821-EF6CFF2551A9}', '49950', '1995-05-26 00:00', 'T', 'N', 'F', 'SHEFFIELD', 'SHEFFIELD', 'SOUTH YORKSHIRE', 'A', 'A']
['{9F4F02C2-498D-492E-918E-EF6D00C8B9BB}', '37500', '1995-06-30 00:00', 'S', 'N', 'F', 'NORWICH', 'NORWICH', 'NORFOLK', 'A', 'A']
['{0464259B-66EF-4681-89DD-F2F06FD19F4F}', '82500', '1995-07-26 00:00', 'D', 'N', 'F', 'CHESTERFIELD', 'NORTH EAST DERBYSHIRE', 'DERBYSHIRE', 'A', 'A']
['{A0438BEA-A986-4A49-B03B-F2F0827E52B7}', '30000', '1995-04-28 00:00', 'S', 'N', 'F', 'STOKE-ON-TRENT', 'STOKE-ON-TRENT', 'STOKE-ON-TRENT', 'A', 'A']
>>>
```

Following business problems we solved using pyspark.

- Which one people prefer to buy a newly built property or an established residential building?
  new = rdd3.filter(lambda a : "Y" in a)
  new.count()
  old = rdd3.filter(lambda a : "N" in a)
  old.count()

```
3
>>> new = rdd3.filter(lambda a : "Y" in a[4])
>>> new.count()
2817007
>>> old = rdd3.filter(lambda a: "N" in a[4])
>>> old.count()
24633493
>>>
```

We know that Y is for new properties and N is for old properties. Here we can observe that the count of new properties is 2.8 million and as that of old is 24.6 million, from this we can conclude that the old properties or the ones which are previously built are preferred by the people.

- Which property is least preferred within the market to buy?
  trdd = rdd3.filter(lambda a : "T" in a[3])
  trdd.count()
  srdd = rdd3.filter(lambda a : "S" in a[3])
  srdd.count()
  drdd = rdd3.filter(lambda a : "D" in a[3])
  drdd.count()
  ordd = rdd3.filter(lambda a : "O" in a[3])
  ordd.count()
  frdd = rdd3.filter(lambda a : 'F' in a[3])
  frdd.count()

```
>>>
>>> trdd = rdd3.filter(lambda a : "T" in a[3])
>>> trdd.count()
8260440
>>> srdd = rdd3.filter(lambda a : "S" in a[3])
>>> srdd.count()
7522784
>>> drdd = rdd3.filter(lambda a : "D" in a[3])
>>> drdd.count()
6333026
>>> ordd = rdd3.filter(lambda a : "O" in a[3])
>>> ordd.count()
405227
>>> frdd = rdd3.filter(lambda a : 'F' in a[3] )
>>> frdd.count()
4929023
>>>
```

  Here we observed that other type of properties with count 405K are least preferred to buy followed by flats (F) with count 4.9 million.

- How many properties are owned by the buyer (freehold)?
  frrdd = rdd3.filter(lambda a : 'F' in a[5])
  frrdd.count()

```
>>> frrdd = rdd3.filter(lambda a : 'F' in a[5] )
>>> frrdd.count()
20996240
>>>
```

  There are total of 20.9 million freehold properties.

2.5 Analysis using Hive

Big data involves processing massive amounts of diverse information and delivering insights. Hadoop is one of the most popular software frameworks designed to process and store big data information. Hive is a tool designed for use with Hadoop.

The Apache Hive data warehouse software facilitates reading, writing, and managing large datasets residing in distributed storage using SQL.

First we loaded data into hive from S3, for that command used is :-

```
hadoop distcp s3://projectg4/pp-complete.csv /project
hadoop distcp s3://projectg4/txns1.txt project/
```

DistCp (distributed copy) is a tool used for large inter/intra-cluster copying. It uses MapReduce to effect its distribution, error handling and recovery, and reporting. It expands a list of files and directories into input to map tasks, each of which will copy a partition of the files specified in the source list.

Here distcp directly loads data from S3 to Hive, no need to load data on local and then local to hdfs.

Now, we create a database and use it :

```
create database project;
use project;
```

To display current database :

```
set  hive.cli.print.current.db=true;
```

We need to create table for analysing data. Table name is ‘pricepaid’ :

```
create external table pricepaid
(TID string, Price bigint, Date_of_Transfer string,
Property_Type string, Old_New string, Duration string,
Town_City string, District string, County string,
PPDCategory_Type string, Record_Status string)
row format delimited
fields terminated by ','
stored as textfile
location '/home/hadoop/project/'
TBLPROPERTIES ("skip.header.line.count"="1");
```

Load the data into table :-

1. We can load data directly from S3.

   load data inpath 's3://projectg4/pricepaid.csv' into table pricepaid
2. Load data from hdfs to table .

   load data inpath '/user/hadoop/project/pricepaid.csv' into table pricepaid

```
hive> Create database project;
OK
Time taken: 1.17 seconds
hive> Use project;
OK
Time taken: 0.062 seconds
hive> set  hive.cli.print.current.db=true;
hive (project)> create external table pricepaid
              > (TID string, Price bigint, Date_of_Transfer timestamp, Property_Type string, Old_New string, Duration string, Town_City string, District stri
ng, County string, PPDCategory_Type string, Record_Status string)
              > row format delimited
              > fields terminated by ','
              > stored as textfile
              > location '/home/hadoop/project/'
              > TBLPROPERTIES ("skip.header.line.count"="1");
OK
Time taken: 0.311 seconds
hive (project)>
              > load data inpath '/user/hadoop/project/pp-complete.csv' into table pricepaid
              > ;
FAILED: SemanticException Line 1:17 Invalid path ''/user/hadoop/project/pp-complete.csv'': No files matching path hdfs://ip-172-31-19-148.ec2.internal:8020/u
ser/hadoop/project/pp-complete.csv
hive (project)> load data inpath '/user/hadoop/project/pricepaid.csv' into table pricepaid;
Loading data to table project.pricepaid
OK
Time taken: 0.985 seconds
hive (project)> select * from pricepaid limit 5;
OK
{3DD6D1E9-68E3-4646-8682-FD7DB9884D97}  76000   NULL    T       N       F       BASINGSTOKE     BASINGSTOKE AND DEANE   HAMPSHIRE       A       A
{82899FB9-496F-4FA3-8249-F6811A87F376}  48000   NULL    T       N       F       LANCASTER       LANCASTER       LANCASHIRE      A       A
{B7B48D70-7FEB-437B-9EAC-F681547AB62F}  59000   NULL    S       N       F       PLYMOUTH        PLYMOUTH        DEVON   A       A
{B75382B6-4B58-4284-A890-FD7E1C297E5D}  69950   NULL    S       N       F       TUNBRIDGE WELLS TUNBRIDGE WELLS KENT    A       A
{B6F968FF-CEB2-4A18-B212-EF6CFE08958D}  57850   NULL    S       Y       F       HEBDEN BRIDGE   CALDERDALE      WEST YORKSHIRE  A       A
Time taken: 1.674 seconds, Fetched: 5 row(s)
hive (project)>
```

When a job is running in Hive, it shows message like give below:

```
hive (project)> select county, count(*) as cnt from pricepaid where duration = 'L'  group by county order by cnt desc;
Query ID = hadoop_20220918063714_84add53a-79d5-40ec-a5f3-2b0aeebd4175
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1663475362145_0003)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1            container       RUNNING      1          0        1        0       0       0
Reducer 2        container        INITED     12          0        0       12       0       0
Reducer 3        container        INITED      1          0        0        1       0       0
--------------------------------------------------------------------------------
VERTICES: 00/03  [>>--------------------------] 0%    ELAPSED TIME: 12.76 s
--------------------------------------------------------------------------------
```

Below are the business problems which we solved using Hive Query language (HQL).

- How many properties are leased per county?

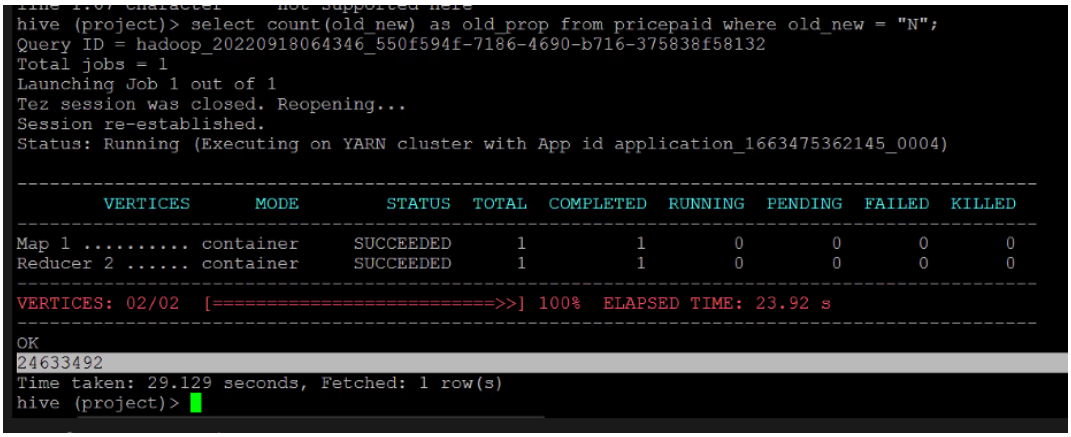
```
select County, count(*) as cnt from pricepaid where duration =
'L' group by county order by cnt desc;
```

```
--------------------------------------------------------------------------------
OK
GREATER LONDON  1836767
GREATER MANCHESTER      647577
LANCASHIRE      217554
WEST MIDLANDS   211068
MERSEYSIDE      176524
SURREY  155889
HERTFORDSHIRE   153680
WEST YORKSHIRE  147942
SOUTH YORKSHIRE 147761
```

Here we can see that Greater London has the highest count of properties leased which is 1.8 million. And some of other properties are also shown in the screenshot.

- How many properties are previously built?

```
select count(old_new) as old_prop from pricepaid where old_new
= 'N'
```



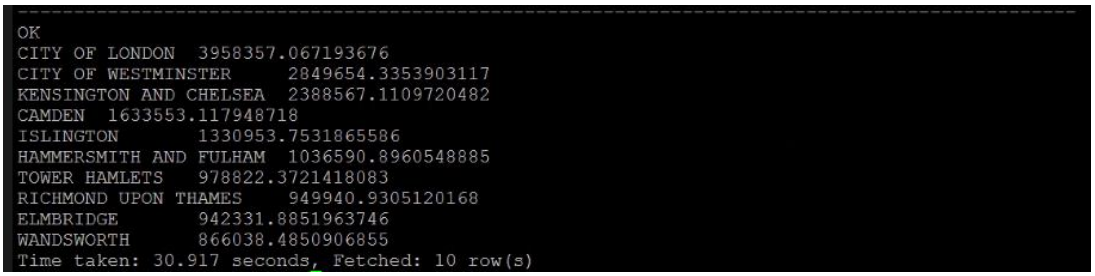
We can observe that 24.6 million properties are previously built out of around 27 million properties.

- What was the average price for each district in the year 2021?

```
select district,avg(price) as avg from pricepaid where
substring(date_of_transfer,1,4)= "2021" group by district
order by avg desc limit 10;
```

OR

```
select district,avg(price) as avg from pricepaid where
date_of_transfer like "2021%" group by district order by avg
desc limit 10;
```



Top ten districts with total average prices in 2021 are listed above out of which London has the highest total average price of £3.95 million.

- What is the average price by property type?

```
select property_type, avg(price) as avg from pricepaid group
by property_type order by avg desc ;
```

In the screenshot give below we can the average price per property type and the other property type has the highest average price of £1.2 million.

```
OK
O       1204578.8970897794
D       284492.7174912909
F       197842.1876288668
S       170190.12181527476
T       155619.07430525182
Time taken: 36.501 seconds, Fetched: 5 row(s)
hive (project)>
```

- Which town/city has maximum growth in real estate investment?

```
select town_city, avg(price) as avg from pricepaid group by
town_city order by avg desc;
```

```
hive (project)> select town_city, avg(price) as avg from pricepaid group by town_city order by avg desc;
Query ID = hadoop_20220918073612_62ad3f68-2386-4481-817d-41aa293c0585
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1663475362145_0008)

--------------------------------------------------------------------------------
        VERTICES      MODE        STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 ..... container     SUCCEEDED      3          3        0        0       0       0
Reducer 2 ..... container     SUCCEEDED      9          9        0        0       0       0
Reducer 3 ..... container     SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 03/03  [==========================>>] 100%  ELAPSED TIME: 36.59 s
--------------------------------------------------------------------------------
OK
GATWICK 2.8232811583333332E7
THORNHILL       985000.0
VIRGINIA WATER  893365.5723524656
CHALFONT ST GILES       818900.8505079825
COBHAM  715594.8597222222
BEACONSFIELD    684180.571733713
ESHER   634516.0959568733
KESTON  604416.3226014166
GERRARDS CROSS  596868.3567996862
ASCOT   555168.623270952
WEYBRIDGE       546820.2736152324
RADLETT 534315.0486111111
BROCKENHURST    527890.8200677392
WINDLESHAM      526862.4686706181
HENLEY-ON-THAMES        520794.08456322295
EAST MOLESEY    515682.65689839574
HARTFIELD       513527.0516605166
ST ALBANS       489715.58821199683
LEATHERHEAD     487173.1023090893
SALCOMBE        486920.028889806
HARPENDEN       483695.4480018973
MUCH HADHAM     477861.513064133
LONDON  469893.77055290295
BURFORD 465158.27138964576
STAINES-UPON-THAMES     464494.8971962617
```

- Top 50 most valued Towns on basis of investments done

```
select town_city, dense_rank() over ( order by avg(price))
from pricepaid group by town_city limit 50;
```

```
OK
GATWICK 1
THORNHILL       2
VIRGINIA WATER  3
CHALFONT ST GILES       4
COBHAM  5
BEACONSFIELD    6
ESHER   7
KESTON  8
GERRARDS CROSS  9
ASCOT   10
```

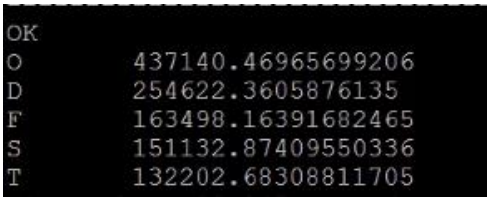
We have ranked top 50 towns on basis of average price and have shown only top 10 here, the town with highest average price is Gatwick.

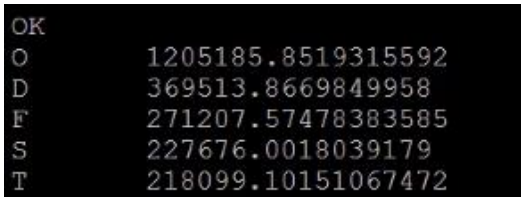
- List of new investments that were done in 2021

```
select property_type, count(*) from pricepaid where
old_new="Y" and date_of_transfer like "2021%" group by
property_type;
```

- What was the average price of each property type in the last two decades?
2000-2009

```
select property_type, avg(price) as avg from pricepaid where
date_of_transfer  like "200%"  group by property_type;
2010-2019
select property_type, avg(price) as avg from pricepaid where
date_of_transfer  like "201%"  group by property_type;
```





We have the average price of each property type of the decade 2000-2009 on left and 2010-2019 on right. We can observe that prices of all property types are increased over the decades.

- Compare average sale price for the last two decades.
2000-2009

```
select  avg(price) as avg from pricepaid where
date_of_transfer  like "200%";
2010-2019
```

2010-2019

```
select avg(price) as avg from pricepaid where date_of_transfer
like "201%" ;
```

We observed that the decade 2000-2009 had average price £170k and the decade 2010-2019 had the average price £298K, so we can say that there is about 100k hike in average prices over the decades.

Chapter III : DATA VISUALISATION

3.1 Introduction to Power BI

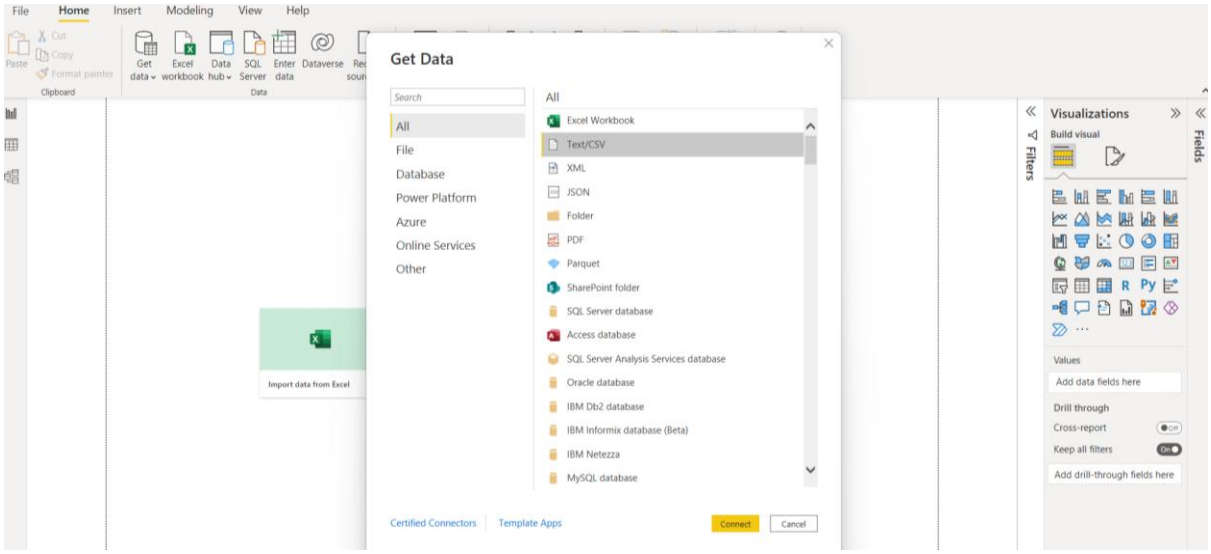
Power BI is a Data Visualization and Business Intelligence tool that converts data from different data sources to interactive dashboards and BI reports. Power BI suite provides multiple software, connector, and services - Power BI desktop, Power BI service based on SaaS, and mobile Power BI apps available for different platforms. These set of services are used by business users to consume data and build BI reports.

Why is visualization important because with the technology revolution data went from expensive ,difficult to find and to abundant ,cheap data to store, understand and analyse the data. Microsoft Power BI is a data visualization tool that allows you to quickly connect your data, prepare it, and model it as you like. It gives a professional environment allowing you to acquire analytics and reporting capabilities. You can publish these reports, therefore allowing all the users to avail the latest information. It gives you the power to transform all your data into live interactive visuals, create customized real time business view dashboards, thus extracting business intelligence for enhanced decision making

Here we are using the clustered columns chart ,donut chart , card , map and slicer

3.2 Importing Data into Power BI

To create dashboard in Power BI, you need to add all data sources in Power BI new . To add a data source, go to the Get data option. Then, select the data source you want to connect and click the Connect button.



3.3 Visuals used for Dashboard

We used following visuals for visualisation.

Clustered column chart : Clustered Bar charts display values (i.e. measures) where the length of the bar or column is proportional to the data. Here we displayed Price by year and Average price by year.

Maps: You can also use a map to view your sales in different places. Choose the globe icon from the visualization panel. Here we displayed price according to district and county.

Donut Charts: Pie charts are quite well known however when it comes to displaying a lot of data. Here we displayed County and price column.

Line Charts: Let's visualize our data using a line chart. Select the line chart from the visualization pane and select the Sales field from Geo Data in the Fields pane.

Slicer : Used to display commonly used or important filters on the report canvas for easier access. Here we used slicer to show Property-type column, Duration column and Old/New column.

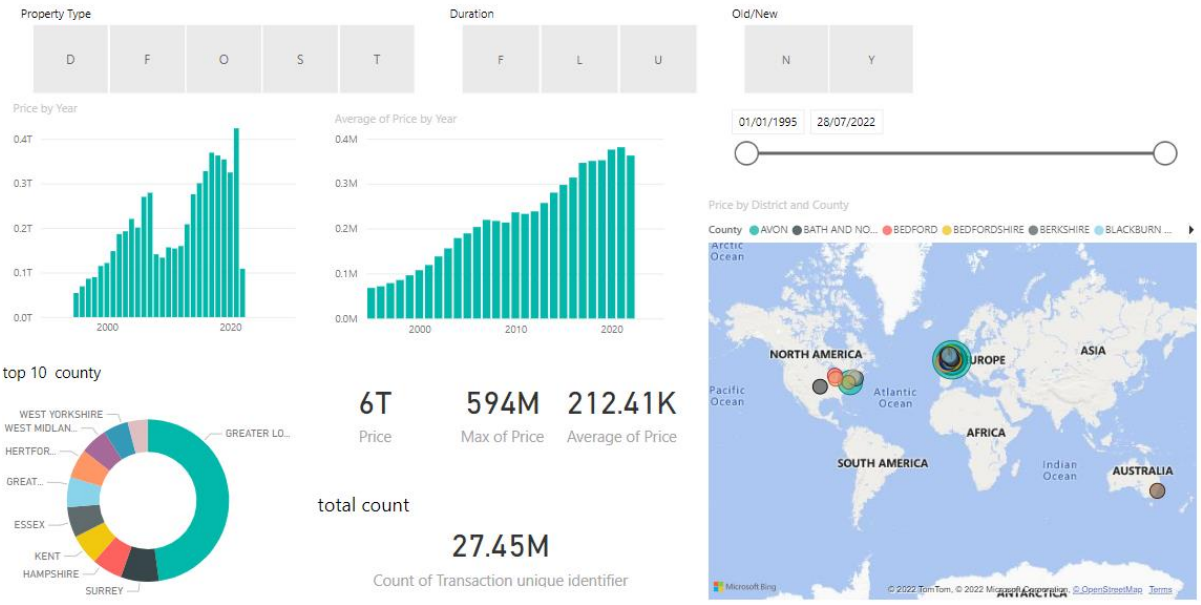
Card : A card is useful to display a single number (or metric value). Here we displayed Price, Maximum Price, Average of price, and total count of Transaction-unique-identifier column.

Pie Chart: Pie charts in Power BI are mostly used for visualizing the percentage contribution for various categories in order to assess the performance of these categories. The pie charts are handy tools as they allow very quick and effective decision making owing to the insights they provide.

Bar and column chart: Bar and Column Charts are one of the most common ways to visualize data. Both of these data visuals use rectangular bars where the size of the bar is proportional to the data values.

Zoom Slider: Zoom Slider is a new feature from Power BI that gives users an easy way to examine a smaller range of data in a chart without having to use a filter.

3.4 Dashboard 1



Dashboard for Price Paid Data

3.4.1 Insights gathered from the Price Paid Dashboard:

We want the information of the top 10 counties. Which county most prefers for living and investment.

Here we use the slicer for the choosing option where we can select which kind of the property type is required.

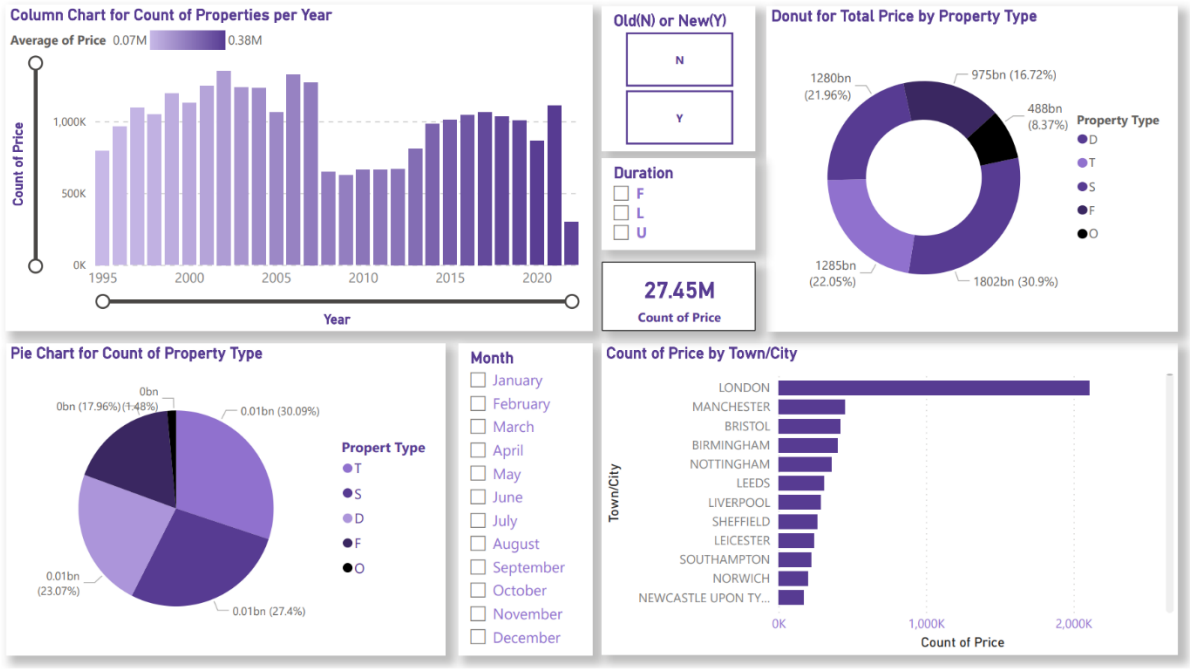
Clustered bar chart showing the average price per year and and showing the individual per year price we can understand easily how price is growing each year. Which district or county has a low price or high price according to area.

Old Property price has increase in year 2021 whereas price for new property has drastically decreased in 2021.

In year 2020 there was drop in price may be due to corona and in year 2021 there is drastically increase in price.

Using Donut chart we can see that Greater London has the highest property price of all the counties.

3.5 Dashboard 2



Dashboard for count of properties

3.5.1 Insights gathered from the Price Paid Dashboard:

We can observe from the card that total number of properties is around 27.45 million. And from the column chart we can observe that the highest number of properties sold is in year 2002 which is 1.35 million and is followed by the year 2006 with count of properties sold 1.32 million.

In pie chart we can see that the count of properties by property type. The terraced houses highest count of 8.26 million which is 30.09% of total properties and the semi-detached houses have the second highest count of 7.52 million that is 27.4% of total properties.

We have used 3 slicers in the dashboard. First is for selecting if the property is previously built or newly built. Second is to filter the property by freehold, leasehold and unknown. And the third slicer is for selecting the month of the year.

In the donut chart we have sum of prices by property type. If we compare the data from pie chart and donut chart, we can see that the terraced houses have the count of properties sold but the detached houses have the highest total price paid which is 1.8 trillion.

From the bar chart we can observe that the highest number of properties sold is in the city of London which is 2.1 million.

Chapter IV : MACHINE LEARNING

4.1 Introduction to Machine Learning

Machine learning (ML) is a type of artificial intelligence (AI) that allows software applications to become more accurate at predicting outcomes without being explicitly programmed to do so. Machine learning algorithms use historical data as input to predict new output values.

There Are Different Types of machine Learning algorithm based on dataset we choose the correct model

Models Used in Project:

1) Multiple Linear Regression Algorithm
2) Decision Tree Algorithm

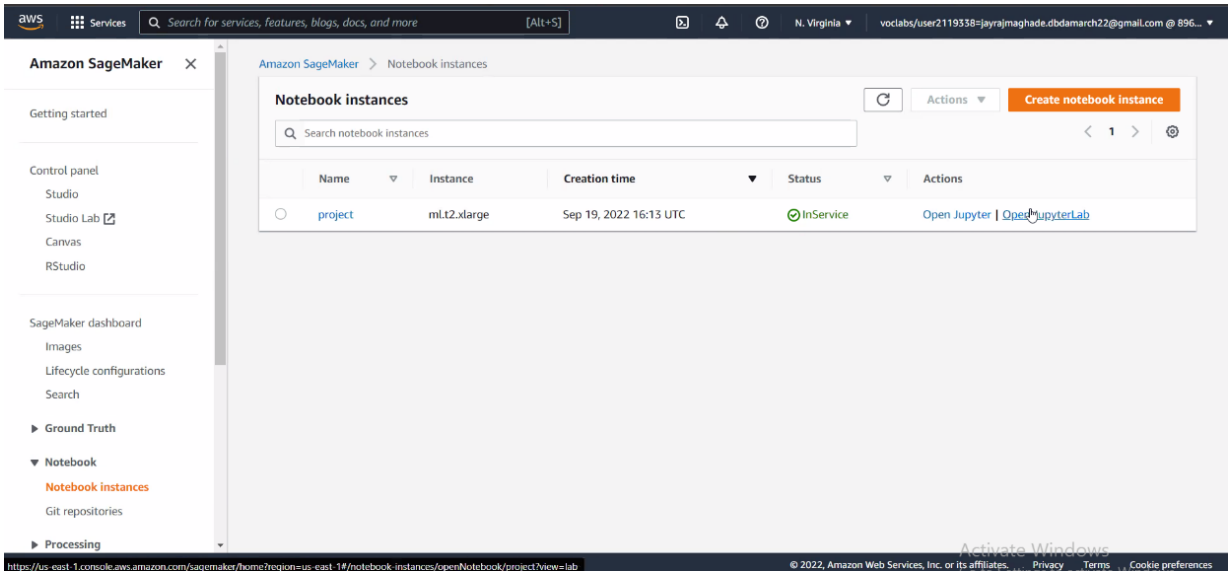
4.2 AWS SageMaker

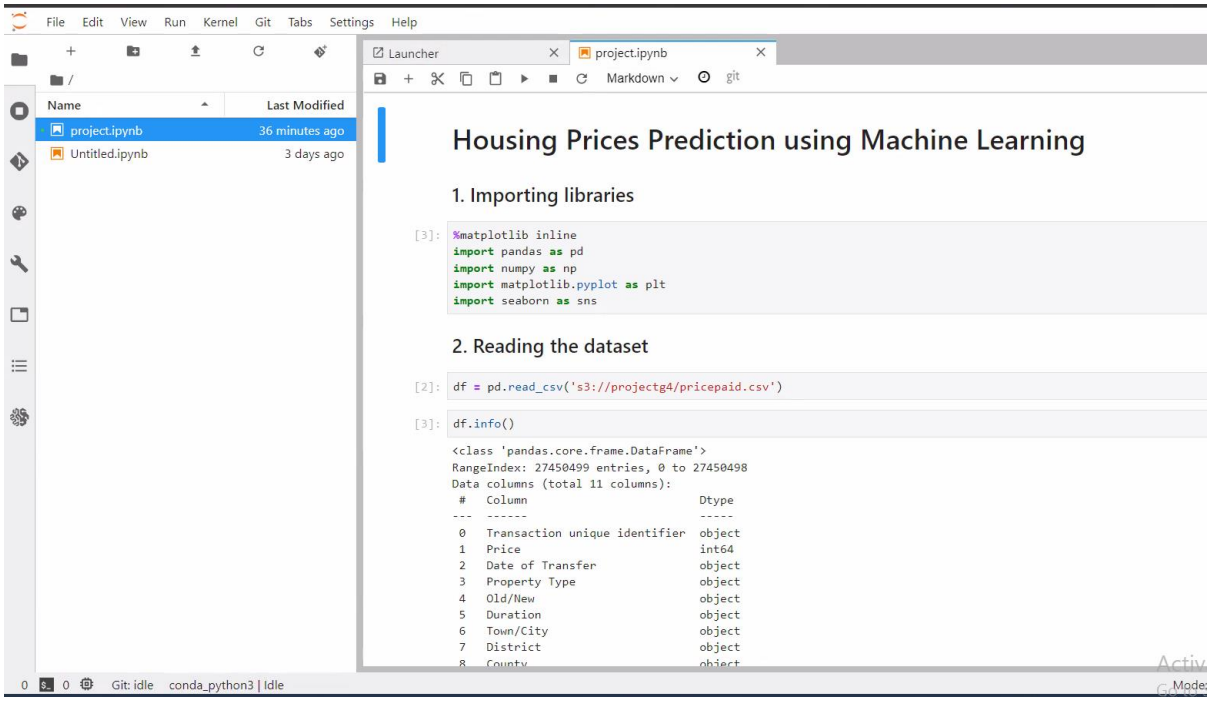
What is SageMaker:

Amazon SageMaker is a **fully managed machine learning service**. With SageMaker, data scientists and developers can quickly and easily build and train machine learning models, and then directly deploy them into a production-ready hosted environment.

For Python EDA & Machine Learning We use AWS service SageMaker. This provides Interface for Python Jupyter Notebook.

Before SageMaker we have to create the EMR Cluster from AWS Console.





Here we read the Data from AWS S3 Storage. And started the EDA.

4.3 EDA And Data Pre-processing:

4.3.1 Introduction To EDA

What is EDA?

Exploratory Data Analysis refers to the critical process of performing initial investigations on data so as to discover patterns, to spot anomalies, to test hypothesis and to check assumptions with the help of summary statistics and graphical representations.

Need Of EDA Before Machine Learning?

Helps you prepare your dataset for analysis. Allows a machine learning model to predict our dataset better. Gives you more accurate results. It also helps us to choose a better machine learning model.

For EDA we use Python Libraries:-

1. Pandas
2. NumPy
3. Matplotlib
4. Seaborn

Before EDA we need to check following parameter:

1. NULL values in Dataset
2. Outliers in Dataset
3. Duplicate Values
4. Multicollinearity

4.3.2 - Operation Perform on EDA

We Drop the Columns which is not Much important as per analysis

- Transaction unique identifier : Not co-related with the price column

- PPD Category Type : It is similar to duration column.

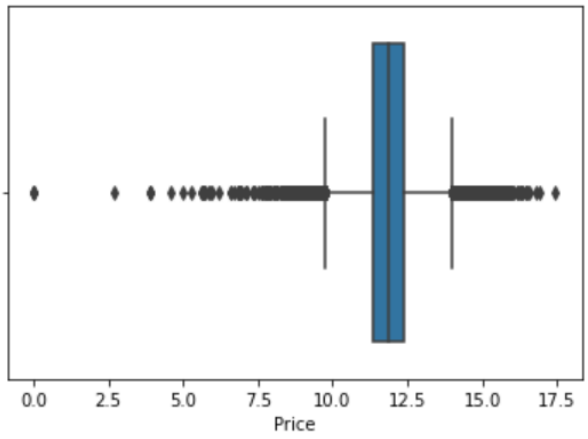
- Record Status : It contains monthly file only Indicates additions, changes and deletions to the records so not necessary.

EDA operations perform for Analysis the dataset:-

1) Unique values present in Columns
2) Separately Analyse the 2021 data
3) Month wise prices in 2021
4) Removing Outliers in Price columns :-There are many outliers in the price column.

```
sns.boxplot(data = df2, x = 'Price')
```

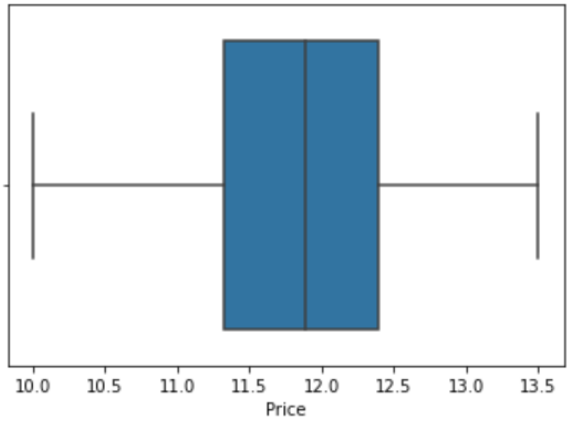
```
<AxesSubplot:xlabel='Price'>
```



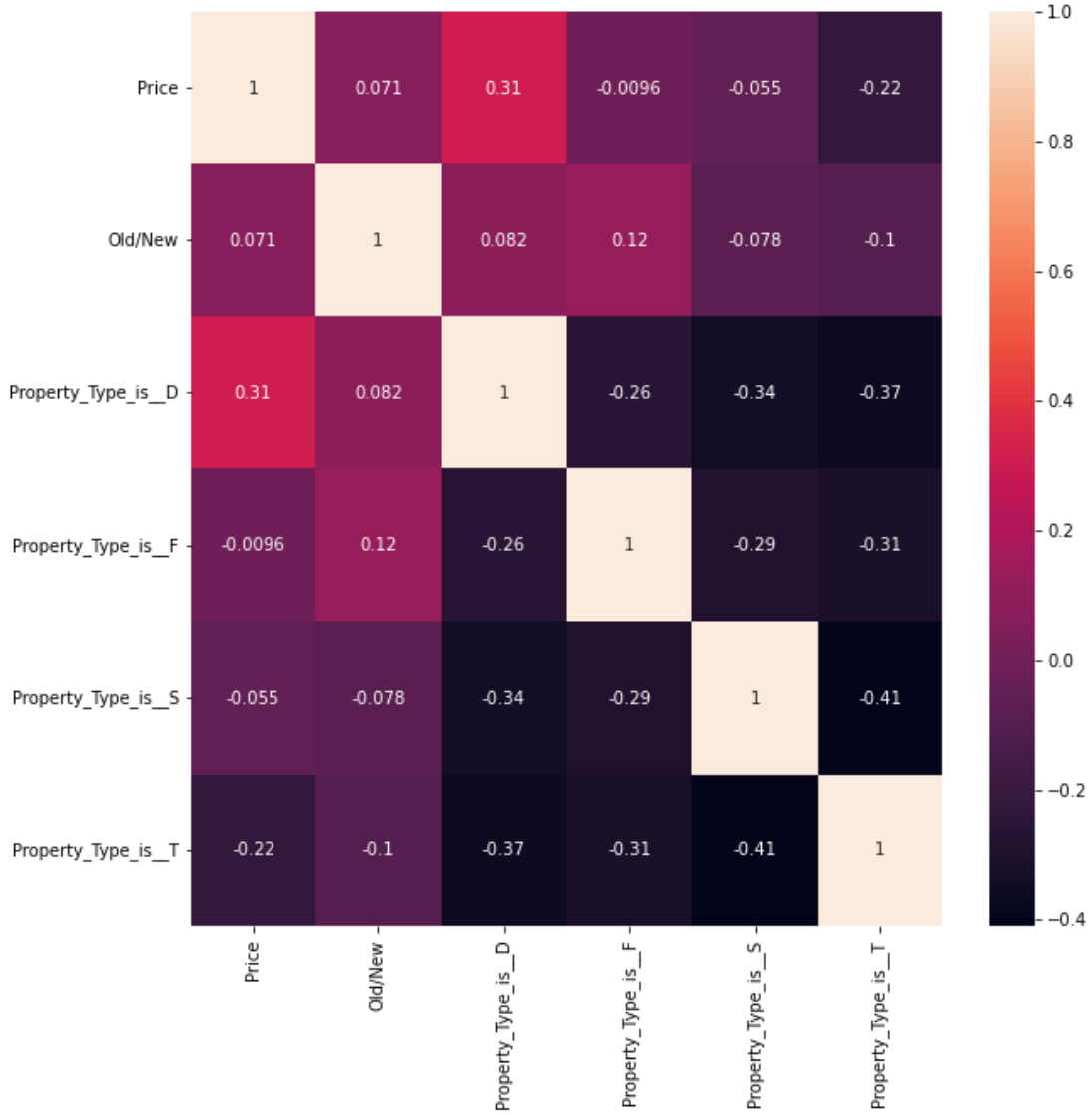
5) Checking outliers with the help of box plot and the removing them from the data.

```
sns.boxplot(data = df2, x = 'Price')
```

```
<AxesSubplot:xlabel='Price'>
```



6) Plotting Heatmap for the co-relation between features



4.3.3 One-Hot-Encoding

One hot encoding is **one method of converting data to prepare it for an algorithm and get a better prediction**. With one-hot, we convert each categorical value into a new categorical column and assign a binary value of 1 or 0 to those columns. Each integer value is represented as a binary vector. We convert all the categorical data to numerical format

4.4 Preparing for the Machine Learning Model

4.4.1 Linear-Regression Model

Points to be remember before using linear model:

Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables.

Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (y) variables, hence called as linear regression.

Mathematically, we can represent a linear regression as:

$y = a_0 + a_1x + \varepsilon$

Y = Dependent Variable (Target Variable)
X = Independent Variable (predictor Variable)
a_0 = intercept of the line (Gives an additional degree of freedom)
a_1 = Linear regression coefficient (scale factor to each input value).
ε = random error

So in our scenario Our **Y** is Target variable means our **Price Column**

X is the remaining all the features which we have to be predicted.

We use multiple Linear Regression because If more than one independent variable is used to predict the value of a numerical dependent variable.

4.4.1.1 Splitting The data into test train using Sklearn

Python-Library to For the Machine learning is sklearn. Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python. It provides a selection of efficient tools for machine learning and statistical modeling including classification, regression, clustering and dimensionality reduction via a consistence interface in Python.

Split the data set into two pieces — a training set and a testing set. This consists of random sampling without replacement about 80 percent of the rows (you can vary this) and putting them into your training set. The remaining 20 percent is put into your test set. ("X_train," "X_test," "y_train," "y_test") for a particular train test split.

4.4.1.2 Rescale variables via standardization

For this we need to import StandardScaler from sklearn.preprocessing.

Python sklearn library offers us with StandardScaler() function to **standardize the data values into a standard format.**StandardScaler removes the mean and scales each feature/variable to unit variance. This operation is performed feature-wise in an independent way.

4.4.1.3 Train and Fit the model and 5-fold cross-validation

K-fold cross-validation is a superior technique to validate the performance of our model. It evaluates the model using different chunks of the data set as the validation set.

We divide our data set into K-folds. K represents the number of folds into which you want to split your data. If we use 5-folds, the data set divides into five sections. In different iterations, one part becomes the validation set.

Train the model on the training set. This is "X_train" and "y_train"

4.1.1.4 Testing and Prediction of model

Test the model on the testing set ("X_test" and "y_test" in the image) and evaluate the performance.

The Accuracy of the model is Near By 46% Which is too low that's why by this conclusion we have to change the model of dataset by another algorithm.

4.4.2 Applying Decision Tree Model

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome.**
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node.** Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and based on the comparison, follows the branch and jumps to the next node.

The Steps for decision tree as per linear regression we only import DecisionTreeRegressor

- **Fitting a Decision-Tree algorithm to the Training set**
- **Predicting the test result**
- **Test accuracy of the result(Creation of Confusion matrix)**

4.5 Conclusion from machine learning:

Accuracy And Prediction:

After applying Decision tree Algorithm we got the nearby 60% accuracy. Which average accuracy. But not good for actual use.

Because applying model on 27 million is big task so we consider only 5 million of data. And model was trained on the same.

That given data. We also Predict the user defined values which is nearly similar to our data.

Github link for Python Notebook

Chapter V : CONCLUSION

From this project we learnt and used various Amazon Web Services like EMR, Sagemaker. We did hands on with Pyspark, Hive, EC2, S3. We did data analysis using the PySpark and Hive and answered some business questions.

For visualisation we used Power BI Desktop and gathered many insights from the data easily with the help of dashboards we created.

After that we performed EDA using Jupyter notebook in SageMaker. Then we used 5 million rows out of 27 million for the prediction purpose. For prediction we used linear regression model and decision tree regressor. We couldn’t achieve decent accuracy, but we got to learn many things from this project.